CRUD

CRUD de Contatos

C	R	U	D
Criar	Listar	Editar	Excluir

React Native | Expo | TypeScript | AsyncStorage

O que e CRUD?

CRUD e a sigla das 4 operacoes basicas de qualquer sistema que trabalha com dados. Seja um app de contatos, uma loja, um sistema escolar -- tudo usa essas 4 acoes:

Letra	Operacao	O que faz no app
`C - Create`	Criar	Preenche o formulario e salva um novo contato
`R - Read`	Ler	Mostra a lista de contatos na tela
`U - Update`	Atualizar	Clica em Editar, muda os dados e salva
`D - Delete`	Excluir	Remove um contato da lista

> **Dica Por que aprender CRUD?**
> Quase todo app do mundo real e um CRUD com uma roupagem diferente. Instagram: CRUD de fotos. WhatsApp: CRUD de mensagens. iFood: CRUD de pedidos. Aprender CRUD e aprender o esqueleto de 90% dos apps que existem.

Passo 1 -- Criar o Projeto do Zero

Abre o terminal na pasta onde voce quer salvar o projeto e roda estes comandos um de cada vez:

1.1 -- Criar o projeto com TypeScript

```
npx create-expo-app crud-contatos -t expo-template-blank-typescript
```

> **Entendendo Por que TypeScript desta vez?**
> TypeScript e JavaScript com tipos. Voce declara que uma variavel e uma string, e o editor avisa se voce tentar colocar um numero nela. Nos capitulos anteriores usamos .js. Aqui usamos .tsx porque e a forma profissional de trabalhar com React Native. Parece complicado no inicio mas e bem simples -- voce vai ver na pratica.

1.2 -- Entrar na pasta

```
cd crud-contatos
```

1.3 -- Instalar o AsyncStorage

```
npx expo install @react-native-async-storage/async-storage
```

> **Entendendo O que e AsyncStorage?**
> E o sistema de armazenamento do React Native. Guarda dados como texto diretamente no celular. Os dados ficam salvos mesmo depois de fechar e reabrir o app. Funciona como o localStorage do navegador, mas para apps mobile.

1.4 -- Abrir no VS Code e rodar

```
code .            <- abre o projeto no VS Code
npx expo start    <- sobe o servidor de desenvolvimento
```

Escaneia o QR Code com o Expo Go no celular. Vai aparecer a tela padrao do Expo.

> **OK Confirmacao antes de continuar**
> O app abre no celular mostrando 'Open up App.tsx to start working on your app!' -- isso significa que o projeto esta pronto. Pode ir para o Passo 2.

Passo 2 -- O Mapa do App

Antes de escrever qualquer codigo, olha para o mapa completo do que vai existir dentro do App.tsx:

```
App.tsx
|
+-- Tipo Contact (TypeScript)
|     id, name, email, phone
|
+-- Estados (useState)
|     contacts[]   <- a lista de contatos
|     name         <- campo nome do formulario
|     email        <- campo email do formulario
|     phone        <- campo telefone do formulario
|     editingId    <- null = criando  /  tem valor = editando
|
+-- Funcoes
|     loadContacts()  <- le os dados salvos quando o app abre
|     saveContacts()  <- salva a lista inteira no celular
|     validate()      <- verifica se os campos estao preenchidos
|     handleCreate()  <- C: cria novo contato
|     handleUpdate()  <- U: atualiza contato existente
|     handleDelete()  <- D: exclui contato
|     startEdit()     <- preenche o form para editar
|     cancelEdit()    <- cancela edicao e limpa o form
|
+-- Tela (return)
      Formulario com 3 campos + botao Salvar/Atualizar
      FlatList mostrando todos os contatos
```

Atencao editingId e o coracao do app

Este e o estado que controla tudo. Quando editingId e null, o botao mostra 'Salvar' e cria um contato novo. Quando editingId tem um valor (o id de um contato), o botao muda para 'Atualizar' e edita aquele contato especifico. Um unico formulario faz as duas coisas!

Passo 3 -- O Codigo Completo

Abre o arquivo App.tsx, apaga TUDO que tem dentro e cola este codigo completo. Depois vamos entender cada parte:

App.tsx -- Parte 1 de 5: Imports e Tipo Contact

```
import React, { useEffect, useMemo, useState } from "react";
import {
  Alert,
  FlatList,
  KeyboardAvoidingView,
  Platform,
  SafeAreaView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
} from "react-native";
import AsyncStorage from "@react-native-async-storage/async-storage";

// Tipo Contact: define a forma de cada contato
// TypeScript garante que todo contato tenha esses 4 campos
type Contact = {
  id: string;
  name: string;
  email: string;
  phone: string;
};

// Chave que identifica os dados no AsyncStorage
const STORAGE_KEY = "@contacts_crud_v1";
```

App.tsx -- Parte 2 de 5: Estados

```
export default function App() {

  // Lista de contatos -- comeca vazia
  const [contacts, setContacts] = useState<Contact[]>([]);

  // Campos do formulario -- comecam vazios
  const [name,  setName]  = useState("");
  const [email, setEmail] = useState("");
  const [phone, setPhone] = useState("");

  // editingId controla o modo do formulario:
  // null     -> modo CRIAR (botao = "Salvar")
  // "123..." -> modo EDITAR (botao = "Atualizar")
  const [editingId, setEditingId] = useState<string | null>(null);

  // isEditing e true quando editingId tem algum valor
  const isEditing = useMemo(() => editingId !== null, [editingId]);
```

App.tsx -- Parte 3 de 5: Storage e Validacao

```
// ===== LER dados salvos no celular =====
async function loadContacts() {
  try {
    const raw = await AsyncStorage.getItem(STORAGE_KEY);
    if (!raw) return;  // ainda nao tem nada salvo
    const parsed: Contact[] = JSON.parse(raw);
    setContacts(parsed);
  } catch (e) {
    Alert.alert("Erro", "Nao foi possivel carregar os dados.");
  }
}

// ===== SALVAR a lista inteira no celular =====
async function saveContacts(next: Contact[]) {
  try {
    setContacts(next);  // atualiza a tela imediatamente
    await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(next));
  } catch (e) {
    Alert.alert("Erro", "Nao foi possivel salvar os dados.");
  }
}

// Roda UMA VEZ quando o app abre -- carrega os dados salvos
useEffect(() => {
  loadContacts();
}, []);

// ===== VALIDACAO dos campos =====
function validate() {
  if (!name.trim())  return "Digite o nome.";
  if (!email.trim()) return "Digite o email.";
  if (!phone.trim()) return "Digite o telefone.";
  if (!email.includes("@")) return "Email invalido (falta @).";
  return null;  // null = tudo ok, pode salvar
}
```

App.tsx -- Parte 4 de 5: Funcoes CRUD

```
// ===== C -- CREATE: criar contato novo =====
async function handleCreate() {
  const err = validate();
  if (err) return Alert.alert("Atencao", err);

  const newContact: Contact = {
    id: String(Date.now()),  // id unico baseado no tempo
    name:  name.trim(),
    email: email.trim(),
    phone: phone.trim(),
  };

  const next = [newContact, ...contacts];  // novo vai para o inicio
  await saveContacts(next);
  setName(""); setEmail(""); setPhone("");  // limpa o formulario
}
```

```
// ===== U -- UPDATE: atualizar contato existente =====
async function handleUpdate() {
  const err = validate();
  if (err) return Alert.alert("Atencao", err);
  if (!editingId) return;

  // .map() percorre a lista e substitui so o contato com o id certo
  const next = contacts.map((c) =>
    c.id === editingId
      ? { ...c, name: name.trim(), email: email.trim(), phone: phone.trim() }
      : c  // os outros ficam iguais
  );

  await saveContacts(next);
  setEditingId(null);  // sai do modo edicao
  setName(""); setEmail(""); setPhone("");
}

// ===== D -- DELETE: excluir contato =====
async function handleDelete(id: string) {
  Alert.alert("Excluir", "Tem certeza que deseja excluir?", [
    { text: "Cancelar", style: "cancel" },
    {
      text: "Excluir",
      style: "destructive",
      onPress: async () => {
        // .filter() cria lista nova SEM o contato com esse id
        const next = contacts.filter((c) => c.id !== id);
        await saveContacts(next);
      },
    },
  ]);
}

// Preenche o formulario com os dados do contato para editar
function startEdit(contact: Contact) {
  setEditingId(contact.id);
  setName(contact.name);
  setEmail(contact.email);
  setPhone(contact.phone);
}

// Cancela a edicao e volta ao modo de criar
function cancelEdit() {
  setEditingId(null);
  setName(""); setEmail(""); setPhone("");
}
```

App.tsx -- Parte 5 de 5: Tela e Estilos

```
return (
  <SafeAreaView style={styles.container}>
    <KeyboardAvoidingView
      style={{ flex: 1 }}
      behavior={Platform.OS === "ios" ? "padding" : undefined}
    >
```

```
        <Text style={styles.title}>CRUD de Contatos</Text>

        {/* ===== FORMULARIO ===== */}
        <View style={styles.card}>
          <Text style={styles.label}>Nome</Text>
          <TextInput
            value={name}
            onChangeText={setName}
            placeholder="Ex: Maria"
            style={styles.input}
          />
          <Text style={styles.label}>Email</Text>
          <TextInput
            value={email}
            onChangeText={setEmail}
            placeholder="Ex: maria@email.com"
            style={styles.input}
            autoCapitalize="none"
            keyboardType="email-address"
          />
          <Text style={styles.label}>Telefone</Text>
          <TextInput
            value={phone}
            onChangeText={setPhone}
            placeholder="Ex: (11) 99999-9999"
            style={styles.input}
            keyboardType="phone-pad"
          />
          <View style={styles.row}>
            {/* Botao muda de texto e funcao dependendo do modo */}
            <TouchableOpacity
              style={[styles.button, isEditing ? styles.btnUpdate :
styles.btnSave]}
              onPress={isEditing ? handleUpdate : handleCreate}
            >
              <Text style={styles.btnText}>
                {isEditing ? "Atualizar" : "Salvar"}
              </Text>
            </TouchableOpacity>
            {/* Botao Cancelar so aparece no modo edicao */}
            {isEditing && (
              <TouchableOpacity
                style={[styles.button, styles.btnCancel]}
                onPress={cancelEdit}
              >
                <Text style={styles.btnText}>Cancelar</Text>
              </TouchableOpacity>
            )}
          </View>
        </View>

        {/* ===== LISTA ===== */}
        <Text style={styles.subtitle}>
          Contatos ({contacts.length})
        </Text>
        <FlatList
          data={contacts}
          keyExtractor={(item) => item.id}
```

```
          contentContainerStyle={{ paddingBottom: 24 }}
          ListEmptyComponent={
            <Text style={styles.empty}>
              Nenhum contato ainda. Salve o primeiro acima!
            </Text>
          }
          renderItem={({ item }) => (
            <View style={styles.item}>
              <View style={{ flex: 1 }}>
                <Text style={styles.itemName}>{item.name}</Text>
                <Text style={styles.itemText}>{item.email}</Text>
                <Text style={styles.itemText}>{item.phone}</Text>
              </View>
              <View style={styles.itemActions}>
                <TouchableOpacity
                  style={[styles.smallBtn, styles.smallEdit]}
                  onPress={() => startEdit(item)}
                >
                  <Text style={styles.smallText}>Editar</Text>
                </TouchableOpacity>
                <TouchableOpacity
                  style={[styles.smallBtn, styles.smallDel]}
                  onPress={() => handleDelete(item.id)}
                >
                  <Text style={styles.smallText}>Excluir</Text>
                </TouchableOpacity>
              </View>
            </View>
          )}
        />
      </KeyboardAvoidingView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container:   { flex: 1, padding: 16, gap: 12, backgroundColor: "#fff" },
  title:       { fontSize: 22, fontWeight: "800", marginBottom: 6 },
  subtitle:    { fontSize: 16, fontWeight: "700", marginTop: 8 },
  card: {
    borderWidth: 1, borderColor: "#e5e5e5",
    borderRadius: 12, padding: 12,
  },
  label:  { fontSize: 12, fontWeight: "700", marginTop: 10, marginBottom: 6 },
  input: {
    borderWidth: 1, borderColor: "#ddd", borderRadius: 10,
    paddingHorizontal: 12, paddingVertical: 10,
  },
  row:    { flexDirection: "row", gap: 10, marginTop: 12 },
  button: { flex: 1, paddingVertical: 12, borderRadius: 10, alignItems: "center"
},
  btnSave:   { backgroundColor: "#111" },
  btnUpdate: { backgroundColor: "#0b5" },
  btnCancel: { backgroundColor: "#666" },
  btnText:   { color: "#fff", fontWeight: "800" },
  empty:     { marginTop: 10, color: "#666" },
  item: {
    flexDirection: "row", gap: 10,
```

```
    borderWidth: 1, borderColor: "#eee",
    borderRadius: 12, padding: 12, marginTop: 10,
  },
  itemName:    { fontSize: 16, fontWeight: "800" },
  itemText:    { color: "#444", marginTop: 2 },
  itemActions: { gap: 8, justifyContent: "center" },
  smallBtn: {
    paddingVertical: 8, paddingHorizontal: 12,
    borderRadius: 10, alignItems: "center",
  },
  smallEdit: { backgroundColor: "#1e88e5" },
  smallDel:  { backgroundColor: "#e53935" },
  smallText: { color: "#fff", fontWeight: "800" },
});
```

OK Salva e testa agora!

O app deve abrir com um formulario no topo e a mensagem 'Nenhum contato ainda' embaixo. Se aparecer assim -- esta funcionando e pode continuar para o Passo 4.

Passo 4 -- Entendendo Cada Parte

App funcionando? Agora vamos entender o que cada bloco faz. Voce nao precisa memorizar -- precisa entender a logica para conseguir adaptar depois.

4.1 -- Tipo Contact (TypeScript)

Isso e TypeScript em acao. O tipo Contact e um contrato -- todo contato do app TEM que ter exatamente esses 4 campos, todos strings (texto).

```
type Contact = {
  id: string;     // identificador unico -- nunca dois iguais
  name: string;   // nome
  email: string;  // email
  phone: string;  // telefone
};

// Se voce tentar criar um contato sem um desses campos,
// o TypeScript avisa com sublinhado vermelho ANTES de rodar.
// Isso evita erros silenciosos que so aparecem em producao.
```

4.2 -- editingId: o estado que controla tudo

Este e o estado mais importante do app. Um unico campo faz o formulario funcionar tanto para criar quanto para editar.

```
// MODO CRIAR (editingId = null):
// -> botao mostra 'Salvar'
// -> onPress chama handleCreate()
// -> botao 'Cancelar' NAO aparece

// MODO EDITAR (editingId = '1718293847123'):
// -> botao mostra 'Atualizar'
// -> onPress chama handleUpdate()
// -> botao 'Cancelar' APARECE

// Na tela, isso se traduz em:
onPress={isEditing ? handleUpdate : handleCreate}
// Se isEditing for true  -> chama handleUpdate
// Se isEditing for false -> chama handleCreate
```

4.3 -- AsyncStorage: salvar e carregar

O AsyncStorage so entende texto. Por isso usamos JSON.stringify para converter o array em texto ao salvar, e JSON.parse para converter de volta ao carregar.

```
// SALVAR (array -> texto):
JSON.stringify([{ id:"1", name:"Maria", email:"m@m.com", phone:"11" }])
// Resultado: '[{"id":"1","name":"Maria","email":"m@m.com","phone":"11"}]'
```

```
// CARREGAR (texto -> array):
JSON.parse('[{"id":"1","name":"Maria"...}]')
// Resultado: [{ id:'1', name:'Maria', email:'m@m.com', phone:'11' }]
```

4.4 -- useEffect: rodar ao abrir o app

O useEffect com array vazio [] roda uma unica vez quando o app abre. E o momento ideal para carregar os dados salvos.

```
useEffect(() => {
  loadContacts();
}, []);
//  ^-- array vazio = roda so uma vez (quando o app abre)

// Se fosse [name]: rodaria toda vez que o campo name mudasse
// Se nao tivesse []: rodaria toda vez que qualquer estado mudasse
```

4.5 -- As 4 operacoes CRUD

C -- handleCreate

Valida os campos, cria o objeto com id unico usando Date.now(), coloca no inicio da lista com spread e salva tudo.

```
const next = [newContact, ...contacts];
//            ^novo primeiro  ^existentes depois

// Date.now() retorna milissegundos desde 1970
// Ex: 1718293847123 -- sempre unico pois o tempo avanca
```

U -- handleUpdate

.map() percorre a lista inteira. Quando encontra o contato com o id certo, substitui os dados. Os outros ficam identicos.

```
const next = contacts.map((c) =>
  c.id === editingId   // este e o que queremos editar?
    ? { ...c, name: name.trim(), email: email.trim(), phone: phone.trim() }
    //  ^ sim: funde os dados novos com os antigos
    : c
    //  ^ nao: fica exatamente igual
);
```

D -- handleDelete

.filter() cria uma lista nova com todos os contatos EXCETO o que tem aquele id.

```
const next = contacts.filter((c) => c.id !== id);
//                                  ^ mantem so os que NAO tem esse id

// Antes:  [{ id:'1' }, { id:'2' }, { id:'3' }]
```

```
// Delete id='2'
// Depois: [{ id:'1' }, { id:'3' }]
```

4.6 -- KeyboardAvoidingView

Quando o teclado aparece no celular, ele cobre parte da tela. KeyboardAvoidingView empurra o conteudo para cima automaticamente para que os campos nao fiquem escondidos.

```
<KeyboardAvoidingView
  behavior={Platform.OS === "ios" ? "padding" : undefined}
>
  {/* conteudo aqui nao fica atras do teclado */}
</KeyboardAvoidingView>

// iOS precisa de 'padding' para funcionar
// Android geralmente ja funciona sozinho (undefined)
```

Passo 5 -- Roteiro de Teste

Com o app rodando no celular, siga esse roteiro para confirmar que as 4 operacoes estao funcionando:

Teste 1 CREATE -- Criar um contato

Preenche os 3 campos com qualquer dado. Toca em 'Salvar'. O contato deve aparecer na lista abaixo. Os campos devem limpar sozinhos apos salvar.

Teste 2 READ -- Ver os dados persistidos

Fecha o app completamente no celular (passa o app para fora da tela). Abre de novo. Os contatos devem continuar aparecendo. Isso confirma que o AsyncStorage esta salvando corretamente.

Teste 3 UPDATE -- Editar um contato

Toca em 'Editar' em qualquer contato. Os campos do formulario devem preencher com os dados daquele contato. O botao deve mudar para 'Atualizar' e aparecer o botao 'Cancelar'. Muda o telefone e toca em 'Atualizar'. O contato na lista deve mostrar o telefone novo.

Teste 4 DELETE -- Excluir um contato

Toca em 'Excluir' em qualquer contato. Deve aparecer um alerta de confirmacao. Confirma. O contato deve sumir da lista.

Passou em tudo? CRUD completo e funcionando!

Voce acabou de construir um app com as 4 operacoes fundamentais de qualquer sistema de dados. Isso e o mesmo esqueleto de apps profissionais do mundo real.

Resumo -- Todos os Comandos

Se precisar comecar do zero, aqui estao todos os comandos na ordem certa:

```
# 1. Criar o projeto com TypeScript
npx create-expo-app crud-contatos -t expo-template-blank-typescript

# 2. Entrar na pasta
cd crud-contatos

# 3. Instalar o AsyncStorage
npx expo install @react-native-async-storage/async-storage
```

```
# 4. Abrir no VS Code
code .

# 5. Rodar o servidor
npx expo start

# 6. Apagar tudo do App.tsx e colar o codigo desta aula
```

Desafios

Primeiro

Organize o projeto por pastas.

Segundo

Tema proprio Use este CRUD como base para um app diferente: lista de filmes, tarefas, produtos, jogos -- troca os campos nome/email/telefone para os campos do seu tema e muda o estilo.

Validacao Valide o telefone: o numero deve ter pelo menos 10 digitos. Pesquise como usar .replace() para remover tudo que nao e numero antes de contar os caracteres.

CRUD funcionando! Parabens!